

마이크로프로세서 기말고사

주의. 단 10문제만 풀 것, 10문제 이상인 경우 더 풀 문제의 점수만큼 감점.

- ① AVR은 외부 장치와 필요한 명령이나 데이터를 주고받을 있도록 연결되어 있다. 이 때 주로 사용하는 방식이 폴링과 인터럽트이다. 두 방법을 설명하고 차이를 설명하라.
- ② PWM(pulse width modulation)이란? 이를 서보모터의 각도 제어 또는 DC 모터의 속도 제어에 사용하는 예를 들어 설명하라.
- ③ 다음 내용을 보고 시리얼 통신의 문제점이 무엇인지를 SPI 통신과 비교하라.

A common serial port, the kind with TX and RX lines, is called "asynchronous" (not synchronous) because there is no control over when data is sent or any guarantee that both sides are running at precisely the same rate. Since computers normally rely on everything being synchronized to a single "clock" (the main crystal attached to a computer that drives everything), this can be a problem when two systems with slightly different clocks try to communicate with each other.

- ④ USART 데이터 프레임의 비트별을 간단하게 설명하시오
- ⑤ AD변환을 위해 필요한 레지스터들의 종류를 나열하고 각 레지스터가 하는 역할에 대해 간략하게 설명하시오
- ⑥ 다음 그림과 같이 구멍이 뚫린 회전원판이 회전할 때 이 원판의 회전 방향을 입력 캡처를 사용하여 알아내는 방법은 무엇인가?



- ⑦ 다음은 INTO의 상승예지를 감지하기 위한 인터럽트 설정이다. 하강예지를 감지하기 위해 수정하라.

```
EMISK |= 1 << INTO; //INT0 인터럽트 활성화
EICRA = 3 << ISC00; //상승예지 인터럽트로 설정
PORTD |= 1 << PDD0; //PDD0(INT0) 핀 내부에 풀업저항 설정
sei(); //전역 인터럽트 활성화
```

8. 어떤 고속PWM을 이용한 RC 서보모터 제어 프로그램 c 코드에서 1도당 OCR2값 변화량 OCR_PER_DEG을 다음과 같이 정의하였다. 위 정의로 나온 결과보다 더 정밀한 RC 서보모터를 위한 프로그램 작성법에 대하여 논하라.

```
#define F_CPU      16000000L
#define PRESCALE 1024L
#define OCR_PER_DEG
((double)((0.0008*F_CPU)/(double)(PRESCALE*80.0)))
```

9. 아날로그 비교기의 인터럽트를 사용하기 위한 절차를 설명하시오.

10. USART의 문자 송신 함수와 문자 수신 함수를 사용하여 printf()와 scanf()의 표준 문자 출력과 표준 문자 입력으로 연결하려 한다. 이 방법을 설명하라.

11. 3개의 LED를 서로 다르게 각 1초, 2초, 4초마다 깜빡이게 하고 싶다. 16비트 타이머/카운터를 이용한 방법을 제시하라

12. RC서보 모터 회전 실험은 1도 당 0.1389 크기의 OCR2 변화가 필요하다. 따라서 정확한 RC 서보모터의 회전을 반영하기에는 다소 부족하다. 프로그램을 개선하여 더 정밀도를 갖는 RC 서보모터 회전 프로그램을 작성하라.

13. 7 세그먼트가 두 개 있다. 데이터 라인은 공통으로 사용하고 별도의 제어라인을 사용하여 두자리 숫자(00-99)를 카운팅할 수 있도록 한다. 원하는 동작이 일어나도록 회로를 설계하고, 제어 코드를 프로그램을 설계하라.

2. PWM(pulse width modulation)이란? 이를 서보모터의 각도 제어 또는 DC 모터의 속도 제어에 사용하는 예를 들어 설명하라.

PWM이란 펄스의 폭을 변조하여 외부 장치를 제어하는 방식을 의미하는데, 이것을 일반 DC 모터의 속도 제어에 사용하는 상황에 대입하여 보자면, 우선 듀티비라는 개념이 필요하다 듀티비는 총 주기 중 펄스가 점유하는 비율을 뜻하는데 이 경우 PWM을 통해서 듀티비가 증가하게 되면 연결된 DC모터의 속도가 빠르게 작동하며, 듀티비가 낮아지면 연결된 DC모터의 속도가 느리게 작동하게 된다. 그리고 PWM을 서보모터에 적용하여 생각해보자면, 서보모터란 PWM의 값을 전압 값으로 바꾸어 -90도에서 90도 범위로 제어하는 경우를 생각해볼 수 있다.

5. AD변환을 위해 필요한 레지스터들의 종류를 나열하고 각 레지스터가 하는 역할에 대해 간략하게 설명하시오

ADMUX : 아날로그 채널을 선택하기 위한 특수 레지스터
ADCSRA : AD 변화 동작에 필요한 제어 설정과 AD 변환이 진행되는 상황을 관찰하는 특수 레지스터
ADCH와 ADCL : 8비트 레지스터인 ADCH와 ADCL을 연결한 16비트 중 10비트에 AD변환값을 가짐

6. 다음 그림과 같이 구멍이 뚫린 회전원판이 회전할 때 이 원판의 회전 방향을 입력 캡처를 사용하여 알아내는 방법은 무엇인가?



포토인터럽트 2개를 활용한다.

포토인터럽트 2개를 고정시키고 구멍에 빛을 통과시킬 때를 High, 통과하지 못할 때를 LOW로 설정하면 시간에 따라 두 포토인터럽트의 HIGH 신호가 들어오는 것을 확인해보면 회전하는 방향을 확인할 수 있다.

이때 주의할 점은 두 개의 포토인터럽트가 두 개의 구멍에 모두 빛을 통과시키는 시점이 존재하게끔 배치하여야 됨

7. 다음은 INT0의 상승에지를 감지하기 위한 인터럽트 설정이다. 하강에지를 감지하기 위해 수정하라.

```
EMISK |= 1<<INT0; //INT0 인터럽트 활성화
EICRA = 3<<ISC00; //상승에지 인터럽트로 설정
PORTD |= 1<<PD0; //PDO(INT0) 핀 내부에 풀업저항 설정
sei(); //전역 인터럽트 활성화
```

EICRA = 2<<ISC00로 수정하면 상승에지에서 하강에지 인터럽트로 설정됨

8. 어떤 고속PWM을 이용한 RC 서보모터 제어 프로그램 c 코드에서 1도당 OCR2값 변화량 OCR_PER_DEG을 다음과 같이 정의하였다. 위 정의로 나온 결과보다 더 정밀한 RC 서보모터를 위한 프로그램 작성법에 대하여 논하라.

```
#define F_CPU 16000000L
#define PRESCALE 1024L
#define OCR_PER_DEG ((double)(0.0008*F_CPU)/((double)(PRESCALE*80.0)))
```

현재의 경우 주파수는 16MHz, 프레스케일러는 1024로 설정되어 있으므로, 주기는 (256X1024)/16M로서 약 16.384ms로 계산된다. 이때 1도당 OCR2의 변화량은 (0.8/90도)/16.384 = OCR2/256으로 계산하면, OCR2의 변화량을 구할 수 있는데 이때 정밀한 서보모터의 구동이란 OCR2의 변화량의 값을 줄이는 것이기 때문에 분모인 16.384[ms]값을 주목해야한다. 결론적으로 OCR2 변화량을 줄여 더욱 정밀하게 컨트롤하기 위해서는 주기의 값이 증가하여야하고, 이는 프레스케일링 값을 증가시키는 방법이나 클럭주파수의 값을 감소시키는 방법을 통해 구현할 수 있다.

9. 아날로그 비교기의 인터럽트를 사용하기 위한 절차를 설명하시오.

1. 아날로그 비교기의 +단자와 -단자에 연결될 신호를 설정한다.
2. 인터럽트가 발생하는 시점을 설정한다.(+값이 크면 ACO에 1 저장 -값이 크면 ACO에 0 저장)
3. 아날로그 비교기 인터럽트 및 전역 인터럽트 활성화
4. 아날로그 비교기 인터럽트 루틴 설정

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile unsigned long timer0;
volatile unsigned int number
```

13. 7 세그먼트가 두 개 있다. 데이터 라인은 공통으로 사용하고 별도의 제어라인을 사용하여 두자리 숫자(00-99)를 카운팅할 수 있도록 한다. 원하는 동작이 일어나도록 회로를 설계하고, 제어 코드를 프로그램을 설계하라.

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile unsigned long timer0;

volatile unsigned int number;
unsigned char led[] = {0x48, 0x7D, 0xC4, 0x64, 0x71, 0x62, 0x43, 0x7c, 0x40, 0x70};

ISR(TIMER0_OVF_vect){
    timer0++;

    if(timer0 % 2 == 0){
        PORTC = (timer0%4 == 0) ? led[(number%100)/10] : led[number%10];
        PORTD = (PORTD | 0xC0) & ~(1 <<((timer0%4 == 0) ? PD7 : PD6));
    }
}

int main(void){
    DDRC = 0xFF;
    DDRD |= 1<<PD7 | 1<<PD6;

    TCCR0 = 1 << CS02 | 1 << CS01;
    TIMSK |= 1 << TOIE0;

    timer0 = 0;
    sei();
    number = 12;

    while(1){}

    return 0;
}
```